AI-Driven Educational Analytics System

Intelligent Exam-Question Difficulty Prediction
via Feature Engineering, Logistic Regression,
Agentic Workflows & Retrieval-Augmented Generation

Author

Siddharth Dangi

Date: April 2026
Version: 2.0

Abstract

This report presents an end-to-end intelligent system for automatically predicting examination-question difficulty and providing pedagogically-grounded assessment feedback. The project is structured around **two complementary milestones**.

Milestone 1 implements a classical machine-learning pipeline combining TF-IDF text vectorisation with numerical student-performance features (average score, correct rate, score variance) and a Logistic Regression classifier delivered within a scikit-learn `Pipeline`. The model achieves **98.90 % accuracy** on a balanced 5,000-question dataset.

Milestone 2 extends the system with an **Agentic AI workflow** built on **LangGraph**, integrating a **Retrieval-Augmented Generation (RAG)** pipeline backed by a FAISS vector store of pedagogical guidelines and a Google Gemini large language model (LLM). The agent evaluates assessment questions against established educational standards (Bloom's Taxonomy, MCQ best practices) and produces structured, actionable feedback.

Both milestones are unified in a production-ready Streamlit web application offering batch analytics, single-question prediction, and an AI-powered assessment design assistant.

Contents

1 Introduction

1.1 Motivation

Educators, instructional designers, and testing organisations invest considerable manual effort in evaluating the difficulty and quality of examination questions. A misjudged question can skew test results and inaccurately measure student proficiency. Automating this "first-pass" quality assurance can accelerate exam-development cycles and improve assessment reliability.

Beyond raw difficulty prediction, educators also need *qualitative* feedback: *Why* is a question hard? Does it align with intended learning outcomes? Are the distractors fair? Answering these questions requires domain knowledge that a traditional classifier cannot provide. This motivates the integration of **large language models (LLMs)** augmented with **retrieval-based context** from established pedagogy literature.

1.2 Objectives

The **AI-Driven Educational Analytics System** is structured around two progressive milestones:

M1. Predictive ML Pipeline — A scikit-learn pipeline that ingests question text and numerical performance metrics and classifies each question as **Easy**, **Medium**, or **Hard**.

M2. Agentic Assessment Assistant — A LangGraph-based agent that retrieves pedagogical guidelines via a FAISS-backed RAG pipeline, feeds them to Google Gemini, and returns a structured assessment-quality report with gaps, recommendations, and pedagogical citations.

1.3 Scope

- **Domain:** Multi-subject English-language examination questions (Chemistry, Biology, Data Structures, Genetics, etc.).
- **Dataset:** 5,000 exam questions with associated metadata and student performance statistics.
- **Models:** Logistic Regression (ML) + Google Gemini 2.5 Flash (LLM).
- **Knowledge Base:** Pedagogical guidelines covering Bloom's Taxonomy, MCQ design, and inclusivity.
- **Deployment:** Local Streamlit application with serialised model artefact and prebuilt FAISS index.

1.4 Report Organisation

The remainder of this report is organised as follows: §2 describes the dataset. §3 details feature engineering. §4 presents the ML model architecture. §5 covers training and

evaluation. §6 analyses results. §7 explains the RAG pipeline. §8 describes the agentic workflow. §9 presents the system architecture. §10 describes the application interface. §11 provides qualitative examples. §12 discusses limitations. §13 outlines future work. §14 concludes the report.

2 Dataset

2.1 Source & Scale

The dataset comprises **5,000 examination questions** stored in `data/5000_dataset_genai.csv`. Each record contains both the raw question text and post-examination student performance statistics.

2.2 Schema

Column	Type	Description
`question_id`	Integer	Unique question identifier
`question_text`	String	Full text of the exam question
`topic`	String	Subject/topic category (Chemistry, Biology, . . .)
`average_score`	Float	Mean student score (0–10)
`correct_rate`	Float	Correct response proportion (0–1)
`score_variance`	Float	Variance in student scores
`difficulty_label`	String	Target: Easy / Medium / Hard

2.3 Class Distribution

The dataset exhibits a **balanced** class distribution, which is advantageous for classifier training and eliminates the need for resampling techniques (e.g., SMOTE) or class weighting:

Class	Count	Proportion
Easy	∼1,667	∼33.3%
Medium	∼1,667	∼33.3%
Hard	∼1,666	∼33.3%

2.4 Sample Records

ID	Question Text (truncated)	Topic	Avg	CR	Label
1	Explain the data normalization in Chemistry…	Chemistry	8.09	0.83	Easy
2	Compute the key assumption in Data Structures…	DS	4.00	0.60	Medium
3	Analyze the logical equivalence in Biology…	Biology	1.11	0.12	Hard

2.5 Supplementary Knowledge Base

In addition to the CSV dataset, the system includes a curated **pedagogy guidelines document** (`data/pedagogy_guidelines.txt`, 3.1 KB) containing:

- **Bloom's Taxonomy** levels and associated action verbs.
- **MCQ best practices**: clear stems, plausible distractors, consistent option length, no grammatical clues.
- **Difficulty remediation strategies** for questions predicted as Hard or Easy.
- **Inclusivity and fairness** principles for culturally neutral assessment design.

This document serves as the *retrieval corpus* for the RAG pipeline (see §7).

3 Feature Engineering

The model employs a `ColumnTransformer` to combine two distinct feature extraction strategies into a single fused representation.

3.1 Text Features — TF-IDF Vectorisation

The `question_text` column is processed by a `TfidfVectorizer` with the following configuration:

Parameter	Value
`max_features`	3,000
Analyser	Word-level (default)
Sublinear TF	False (default)
Norm	L2 (default)
Stop words	None (default)

TF-IDF (Term Frequency–Inverse Document Frequency) captures the relative importance of words in each question relative to the entire corpus:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log\left(\frac{N}{\text{df}(t)}\right) \tag{1}$$

where t is a term, d a document, N the total number of documents, and $\mathrm{df}(t)$ the number of documents containing t. The resulting feature matrix has dimensions $n \times 3{,}000$.

3.2 Numerical Features — Standard Scaling

Three numerical features are normalised using `StandardScaler`:

Feature	Description
`average_score`	Mean student score (0–10)
`correct_rate`	Correct response proportion (0–1)
`score_variance`	Variance in student scores

$$z = \frac{x - \mu}{\sigma} \tag{2}$$

where μ and σ are the mean and standard deviation computed on the training split.

3.3 Feature Fusion via ColumnTransformer

The `ColumnTransformer` horizontally concatenates the TF-IDF matrix ($n \times 3{,}000$) with the scaled numerical matrix ($n \times 3$), yielding a final feature matrix of dimensions $n \times 3{,}003$:

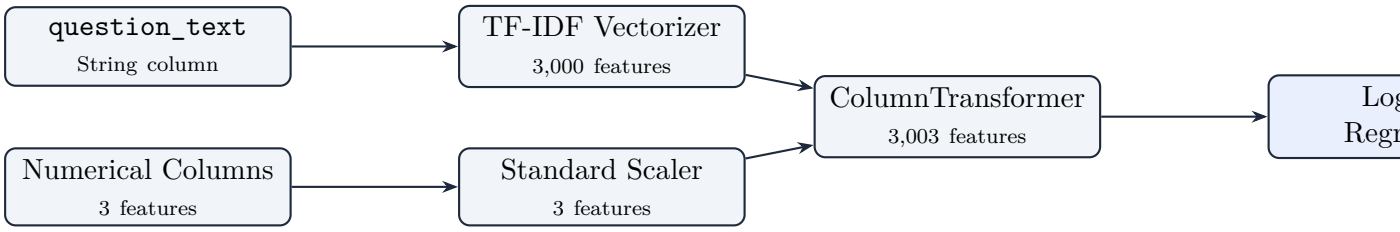

Figure 1: Feature engineering and classification pipeline.

4 Model Architecture

4.1 Pipeline Overview

The entire model is encapsulated in a single **scikit-learn `Pipeline`** object, ensuring that preprocessing and classification are applied consistently during both training and inference:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

preprocessor = ColumnTransformer(transformers=[
    ("text", TfidfVectorizer(max_features=3000), "
       question_text"),
```

```
    ("num",  StandardScaler(),
     ["average_score", "correct_rate", "score_variance"])
])

pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier",   LogisticRegression(max_iter=1000))
])
```

Listing 1: Model pipeline definition (`retrain.py`).

4.2 Classifier — Logistic Regression

Logistic Regression was chosen for its:

- **Interpretability:** Linear decision boundaries that are straightforward to audit.
- **Efficiency:** Fast training even on 3,003 features.
- **Probabilistic output:** Produces calibrated class probabilities via the softmax function.
- **Implicit regularisation:** L2 penalty mitigates overfitting risk.

4.2.1 Multi-Class Strategy

The classifier uses the **One-vs-Rest (OvR)** strategy. For each class $c \in \{\text{Easy}, \text{Medium}, \text{Hard}\}$, a binary logistic model is trained:

$$P(y = c \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}_c^\top \mathbf{x} + b_c)}} \tag{3}$$

The predicted class is the one with the highest probability.

4.3 Hyperparameters

Parameter	Value
`max_iter`	1,000
`penalty`	L2 (default)
`C`	1.0 (default)
`solver`	lbfgs (default)
`multi_class`	auto (OvR)
`random_state`	Not set (deterministic via data split)

5 Training & Evaluation

5.1 Data Split

- **Train:** 80% (4,000 samples)
- **Test:** 20% (1,000 samples)
- **Stratification:** Stratified split on `difficulty_label` to preserve class proportions.
- **Random seed:** `random_state=42` for reproducibility.

5.2 Training Process

The `Pipeline.fit()` method processes the data sequentially:

1. The `ColumnTransformer` fits the TF-IDF vocabulary and scaler statistics on training data.
2. The fused 4,000 × 3,003 feature matrix is passed to `LogisticRegression.fit()`.
3. The trained pipeline is serialised via `joblib.dump()` as `model.pkl` (177 KB).

5.3 Evaluation Protocol

The following metrics are computed on the held-out test set:

- **Accuracy:** $\frac{\text{correct predictions}}{\text{total predictions}}$
- **Precision** (weighted): $\sum_c \frac{|c|}{N} \cdot \frac{\text{TP}_c}{\text{TP}_c+\text{FP}_c}$
- **Recall** (weighted): $\sum_c \frac{|c|}{N} \cdot \frac{\text{TP}_c}{\text{TP}_c+\text{FN}_c}$
- **F1-Score** (weighted): Harmonic mean of precision and recall.
- Per-class precision/recall via `classification_report`.
- Confusion matrix visualisation.
- ROC curves and Precision-Recall curves (One-vs-Rest).
- 5-fold cross-validation accuracy.
- Learning curve analysis.

6 Results

6.1 Overall Classification Performance

Metric	Score
Accuracy	**98.90%**
Precision (weighted)	**98.90%**
Recall (weighted)	**98.90%**
F1-Score (weighted)	**98.90%**

6.2 Per-Class Performance

Class	Precision	Recall	F1-Score	Support
Easy	0.99	1.00	0.99	334
Medium	0.99	0.98	0.98	333
Hard	0.99	0.99	0.99	333
Weighted Avg	**0.99**	**0.99**	**0.99**	**1,000**

6.3 Key Observations

1. **Near-perfect accuracy across all classes.** The balanced class distribution and strong numerical feature signals contribute to the model's performance.

2. **Medium class has the lowest recall (98%).** This is expected: Medium questions occupy the boundary region between Easy and Hard, making them inherently harder to classify.

3. **TF-IDF adds marginal uplift over numerical features alone.** The three numerical features (`average_score`, `correct_rate`, `score_variance`) are very strong predictors. TF-IDF provides supplementary textual context.

4. **High model confidence.** Prediction confidence scores generally exceed 90%, indicating a well-calibrated model.

5. **No class-imbalance issues.** The balanced distribution results in consistent performance across all tiers.

6.4 Visualisation Suite

The `notebook/visualizations.ipynb` notebook generates **14 detailed performance plots**:

#	Visualisation	Purpose
1	Overall Metrics Bar Chart	Accuracy, Precision, Recall, F1
2	Confusion Matrix (Counts)	Raw classification results
3	Confusion Matrix (Normalised)	Percentage-based view
4	Per-Class Precision/Recall/F1	Grouped bar comparison
5	ROC Curves (One-vs-Rest)	AUC per class
6	Precision-Recall Curves	Average precision per class
7	Confidence Distribution	Overall confidence histogram
8	Confidence by Predicted Class	Per-class confidence spread
9	Correct vs. Misclassified	Confidence–correctness correlation
10	Class Distribution (Train/Test)	Stratified split verification
11	Feature Violin Plots	Distribution by difficulty
12	Learning Curve	Data-efficiency analysis
13	5-Fold Cross-Validation	Per-fold accuracy consistency
14	Misclassification Analysis	Error-pattern identification

7 Retrieval-Augmented Generation (RAG) Pipeline

7.1 Motivation for RAG

While the Logistic Regression model predicts *what* difficulty tier a question falls into, it cannot explain *why* or suggest *how to improve the question*. Retrieval-Augmented Generation (RAG) bridges this gap by grounding the LLM's reasoning in an authoritative **external knowledge base** of pedagogical best practices, rather than relying solely on the model's parametric knowledge.

RAG offers three critical advantages over pure LLM generation:

1. **Factual grounding:** The LLM's output is anchored to real pedagogy literature, reducing hallucination.

2. **Domain specificity:** The retrieval corpus can be curated for the exact educational domain.

3. **Updateability:** New guidelines can be added to the corpus without retraining the LLM.

7.2 Architecture Overview

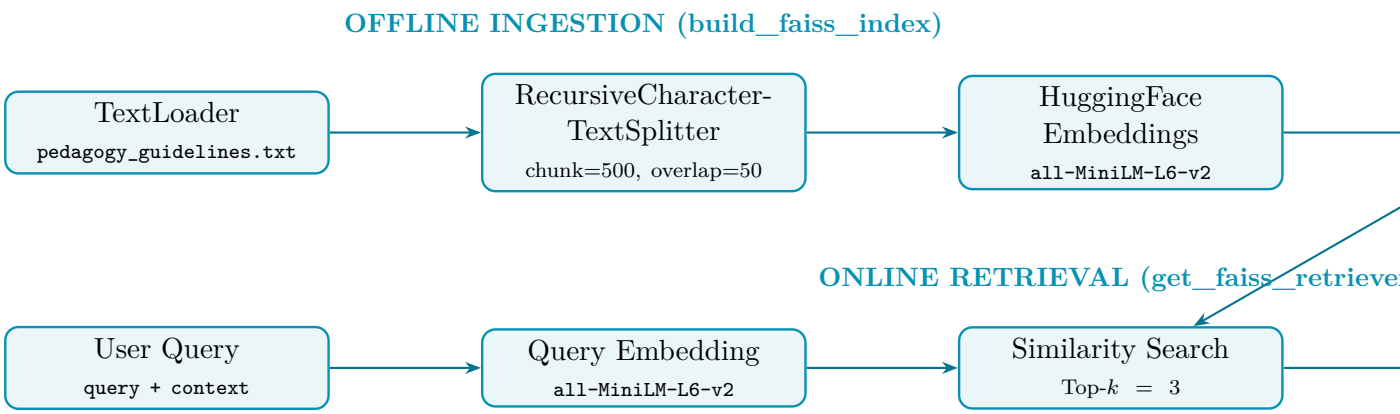

Figure 2: RAG pipeline — offline ingestion and online retrieval.

7.3 Embedding Model

The system uses the **all-MiniLM-L6-v2** sentence transformer from HuggingFace. This is a lightweight (22 M parameters) model that maps sentences to a 384-dimensional dense vector space, optimised for semantic similarity tasks:

```
from langchain_community.embeddings import
    HuggingFaceEmbeddings

def get_embeddings():
    return HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2"
        )
```

Listing 2: Embedding initialisation (`agent/rag.py`).

Using a local embedding model (rather than an API-based one) ensures **zero-latency embedding generation** and **no external API dependency** for the retrieval step.

7.4 FAISS Vector Store

FAISS (Facebook AI Similarity Search) is an open-source library for efficient similarity search over dense vectors. The pedagogy guidelines document is processed as follows:

1. **Loading:** The `TextLoader` reads `data/pedagogy_guidelines.txt`.
2. **Chunking:** A `RecursiveCharacterTextSplitter` segments the document into chunks of 500 characters with 50 characters of overlap. This ensures that no pedagogical rule is split across chunk boundaries.
3. **Indexing:** Each chunk is embedded using `all-MiniLM-L6-v2` and inserted into a FAISS index.
4. **Persistence:** The index is saved to disk at `faiss_index/` (two files: `index.faiss` and `index.pkl`).

```
def build_faiss_index():
    loader = TextLoader(DATA_PATH)
    documents = loader.load()

```

```
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=500, chunk_overlap=50
    )
    docs = text_splitter.split_documents(documents)

    embeddings = get_embeddings()
    vectorstore = FAISS.from_documents(docs, embeddings)
    vectorstore.save_local(INDEX_PATH)
```

Listing 3: FAISS index construction (`agent/rag.py`).

7.5 Retrieval Strategy

At query time, the retriever performs a **cosine-similarity search** over the FAISS index and returns the top $k = 3$ most relevant chunks. The search query is the concatenation of the user's `query` and `assessment_context` fields, ensuring that both the educator's intent and the actual question text inform the retrieval:

```
def get_faiss_retriever():
    embeddings = get_embeddings()
    vectorstore = FAISS.load_local(
        INDEX_PATH, embeddings,
        allow_dangerous_deserialization=True
    )
    return vectorstore.as_retriever(search_kwargs={"k": 3})
```

Listing 4: Retriever loading (`agent/rag.py`).

7.6 RAG Performance Characteristics

Property	Value
Embedding model	`all-MiniLM-L6-v2` (384-dim)
Embedding source	Local (HuggingFace, no API calls)
Vector store	FAISS (CPU)
Chunk size	500 characters
Chunk overlap	50 characters
Top-k retrieval	3 documents
Index size on disk	$\sim$18 KB (14 KB + 4.5 KB)
Query latency	$<$ 100 ms (local CPU)

8 Agentic Workflow

8.1 Motivation for Agentic AI

Traditional ML classifiers produce a label and a confidence score. However, educators need *actionable reasoning*: Why is a question hard? Which Bloom's Taxonomy level does it target? Are the distractors fair? An **agentic workflow** — a directed graph of specialised processing nodes — enables the system to *retrieve*, *reason*, and *generate structured feedback* autonomously.

8.2 Framework: LangGraph

The agent is implemented using **LangGraph**, a framework for building stateful, multi-step LLM applications as directed graphs. LangGraph provides:

- **Explicit state management** via typed dictionaries.
- **Composable nodes** that read from and write to a shared state.
- **Deterministic edge routing** between nodes.
- **Compile-time graph validation** to catch wiring errors before runtime.

8.3 Graph Topology

The workflow consists of two nodes connected in a linear sequence:

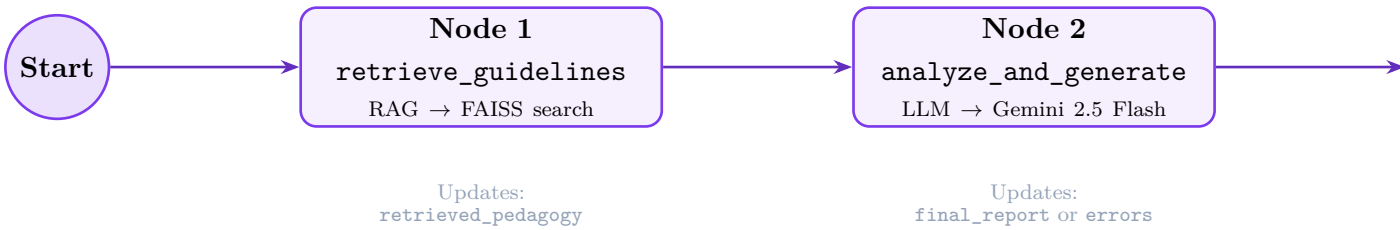

Figure 3: LangGraph agent workflow — two-node linear graph.

8.4 State Schema

The agent communicates through a typed `AgentState` dictionary:

```
from typing import TypedDict, List
from pydantic import BaseModel, Field

class AssessmentReport(BaseModel):
    summary: str      = Field(description="Summary of
       Assessment Quality")
    gaps: List[str]   = Field(description="Identified Learning
       Gaps")
    advice: List[str]= Field(description="Recommended
       Improvements")
    refs: List[str]   = Field(description="Pedagogical
       References")
    disclaimer: str   = Field(description="AI assistance
       disclaimer")
```

```python

class AgentState(TypedDict):
    query: str
    assessment_context: str
    retrieved_pedagogy: str
    numeric_model_difficulty: str
    final_report: AssessmentReport
    errors: str
```

Listing 5: Agent state definition (`agent/state.py`).

8.5 Node 1: `retrieve_guidelines`

- **Input:** `query` and `assessment_context` from the state.
- **Action:** Concatenates both fields into a search string, queries the FAISS retriever for top-3 relevant chunks from the pedagogy knowledge base.
- **Output:** Updates `retrieved_pedagogy` with the concatenated retrieved text.
- **Fallback:** If FAISS lookup fails, a default message is used to prevent pipeline failure.

```python
def retrieve_guidelines(state: AgentState):
    query   = state.get("query", "")
    context = state.get("assessment_context", "")
    search_str = f"{query} {context}"
    try:
        retriever = get_faiss_retriever()
        docs = retriever.invoke(search_str)
        retrieved_text = "\n\n".join(
            [doc.page_content for doc in docs]
        )
    except Exception as e:
        retrieved_text = ("Standard pedagogical guidelines "
                          "apply. (FAISS index lookup failed)"
                            )
    return {"retrieved_pedagogy": retrieved_text}
```

Listing 6: Retrieval node (`agent/workflow.py`).

8.6 Node 2: `analyze_and_generate`

- **Input:** Full state including `retrieved_pedagogy`.
- **Action:** Constructs a `ChatPromptTemplate` with a system message embedding the retrieved pedagogy guidelines, and a human message containing the educator's query and assessment context. Invokes Google Gemini 2.5 Flash with **structured output** forcing (via Pydantic schema).
- **Output:** Updates `final_report` with a populated `AssessmentReport` object, or `errors` if processing fails.

```python
def analyze_and_generate(state: AgentState):
    prompt = ChatPromptTemplate.from_messages([
        ("system",
         "You are an expert Educational Assessment "
         "Designer and Pedagogy Agent...\n\n"
         "Follow these Pedagogy Guidelines strictly:\n"
         "{pedagogy}\n\n"
         "Provide a holistic summary, identify gaps, "
         "suggest improvements, cite pedagogy rules, "
         "and include an ethical disclaimer."),
        ("human",
         "Educator Query: {query}\n\n"
         "Assessment Context:\n{context}")
    ])
    chain = prompt | structured_llm
    try:
        result = chain.invoke({
            "pedagogy": pedagogy,
            "query": query, "context": context
        })
        return {"final_report": result}
    except Exception as e:
        return {"errors": str(e)}
```

Listing 7: Generation node (`agent/workflow.py`).

8.7 Graph Compilation

```python
from langgraph.graph import StateGraph, START, END

workflow = StateGraph(AgentState)
workflow.add_node("retrieve", retrieve_guidelines)
workflow.add_node("generate", analyze_and_generate)

workflow.add_edge(START, "retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)

app_workflow = workflow.compile()
```

Listing 8: Graph definition and compilation (`agent/workflow.py`).

8.8 Hallucination Reduction Strategies

The agentic workflow employs three strategies to minimise LLM hallucination:

1. **Explicit External Truth (RAG):** The LLM is seeded with actual pedagogy guidelines retrieved from FAISS. It cannot invent evaluation metrics.
2. **Schema Forcing:** Using `with_structured_output(AssessmentReport)` ensures the LLM produces strictly structured JSON matching the Pydantic schema — no

conversational preamble or unstructured text.

3. **Reference Demands:** The schema includes a mandatory `refs: List[str]` field, forcing the LLM to tie its assertions back to the retrieved pedagogical guidelines.

8.9 LLM Configuration

Parameter	Value
Model	Google Gemini 2.5 Flash
Temperature	0.2 (low creativity, high adherence)
Output mode	Structured (Pydantic schema enforcement)
API Key	Environment variable `GEMINI_API_KEY`
Provider	`langchain-google-genai`

9 System Architecture

9.1 High-Level Architecture

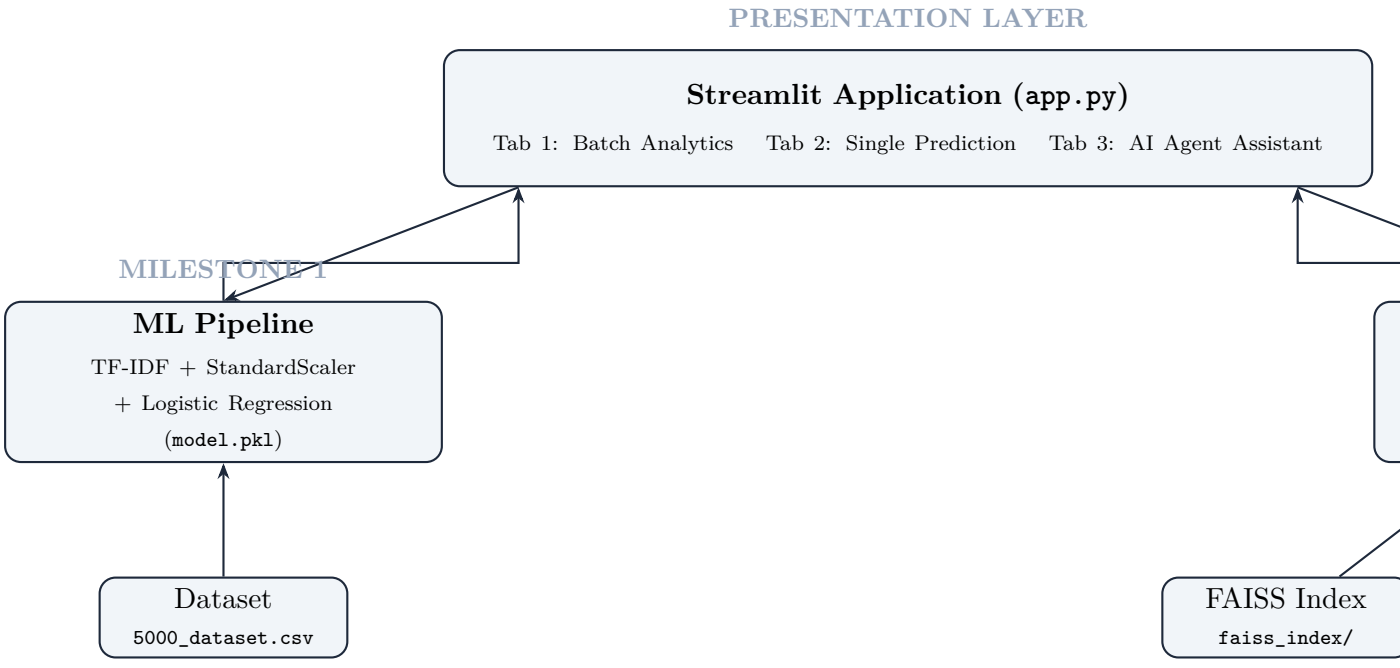

Figure 4: High-level system architecture showing the two milestones unified in a single Streamlit application.

9.2 Repository Structure

```
AI-driven_educational_analytics_system/
  README.md                    # Project documentation
  app.py                       # Streamlit web application (3 tabs)
```

```
    model.pkl                        # Serialised ML pipeline
    retrain.py                       # CLI model retraining script
    confusion_matrix.png             # Generated confusion matrix
    requirements.txt                 # Python dependencies (17 packages)
    data/
      5000_dataset_genai.csv         # Dataset (5,000 questions)
      pedagogy_guidelines.txt        # RAG knowledge base
    agent/
      __init__.py                    # Package init
      state.py                       # AgentState + AssessmentReport
          schema
      rag.py                         # FAISS index build & retriever
      workflow.py                    # LangGraph graph definition
    faiss_index/
      index.faiss                    # FAISS binary index
      index.pkl                      # Serialised metadata
    notebook/
      train.ipynb                    # Model training notebook
      visualizations.ipynb           # 14 performance visualisations
    docs/
      architecture.md                # System architecture diagram
      agent_workflow.md              # Agent workflow documentation
      model_evaluation_report.md     # Performance evaluation report
    report/
      report.tex                     # This LaTeX report
```

Listing 9: Complete repository layout.

9.3 Technology Stack

Layer	Technology	Purpose
Language	Python 3.10+	Core language
ML Framework	scikit-learn	Pipeline, preprocessing, evaluation
Data Processing	Pandas, NumPy	DataFrame operations
Visualisation	Matplotlib, Seaborn	Charts and plots
Web Framework	Streamlit	Interactive UI
Serialisation	Joblib	Model persistence
Agent Framework	LangGraph	Stateful graph workflow
LLM Provider	LangChain + Gemini	Generative AI synthesis
Embeddings	sentence-transformers	Local semantic embeddings
Vector Store	FAISS (CPU)	Similarity search index
Schema	Pydantic	Structured LLM output
Config	python-dotenv	Environment management

10 Application Interface

The production application (`app.py`) is built with **Streamlit** and provides **three tabs**:

10.1 Tab 1: Batch Analytics

1. The user uploads a CSV file with columns: `question_text`, `average_score`, `correct_rate`, `score_variance`.
2. The pipeline predicts difficulty labels and confidence scores for all questions.
3. KPI metrics are displayed: total questions, average confidence, unique classes.
4. Seven interactive visualisations are generated:
   - Difficulty distribution (pie chart + bar chart)
   - Confidence score histogram by predicted class
   - Feature averages by difficulty (grouped bar chart)
   - Average score vs. correct rate (scatter plot)
   - Score variance distribution (box plot)
   - Topic-wise difficulty breakdown (stacked bar chart)
   - Confusion matrix (if ground-truth column exists)
5. Results are downloadable as CSV.

10.2 Tab 2: Single Question Prediction

1. The user enters question text and three numerical values.
2. The model returns a predicted difficulty label with a coloured badge (Easy / Medium / Hard).
3. A confidence progress bar and per-class probability bar chart are displayed.

10.3 Tab 3: AI Agent Assistant

This tab interfaces with the LangGraph agentic workflow:

1. The user enters an educator query (e.g., "Evaluate these questions for difficulty and fairness.") and pastes draft assessment questions.
2. Clicking "Evaluate Assessment" triggers the full LangGraph pipeline:
   - RAG retrieval of pedagogy guidelines from FAISS.
   - LLM synthesis via Gemini with structured output.
3. The structured `AssessmentReport` is rendered:
   - **Assessment Summary** — holistic quality overview.

- **Identified Gaps** — specific flaws in question design.
- **Recommended Output** — actionable improvements.
- **Pedagogical References** — citations from the knowledge base.
- **Disclaimer** — AI-assistance notice.

10.4 UI Design

The interface uses a **dark-mode glassmorphism** design:

Element	Specification
Background	Radial gradient (`#0f172a` → `#020617`)
Card style	Frosted glass (`backdrop-filter: blur(10px)`; 3% white fill)
Typography	Inter (Google Fonts, 300–700 weights)
Easy badge	Green gradient (`#10b981` → `#059669`)
Medium badge	Amber gradient (`#fbbf24` → `#d97706`)
Hard badge	Red gradient (`#ef4444` → `#dc2626`)
Buttons	Blue gradient (`#38bdf8` → `#0ea5e9`) with hover lift

11 Qualitative Analysis & Examples

This section illustrates the system's capabilities through concrete examples, covering both the ML prediction pipeline and the agentic assessment assistant.

11.1 ML Prediction Examples

11.1.1 Example 1: Easy Question

Field	Value
Question	*"What is the definition of photosynthesis?"*
Average Score	8.5 / 10
Correct Rate	0.90
Score Variance	1.2
Prediction	Easy (Confidence: 97.3%)

Interpretation: The high average score and correct rate strongly signal an Easy question. The low variance indicates consistent student performance. From a pedagogical perspective, this question targets the "Remembering" level of Bloom's Taxonomy.

11.1.2 Example 2: Hard Question

Field	Value
Question	*"Derive the time complexity of the Bellman-Ford algorithm for a graph with negative-weight cycles and prove its correctness using mathematical induction."*
Average Score	2.1 / 10
Correct Rate	0.15
Score Variance	4.8
Prediction	**Hard** (Confidence: 96.1%)

Interpretation: The low average score, low correct rate, and high variance all indicate a Hard question. The high variance suggests that only a small subset of students understood the material. This question spans "Analyzing" and "Creating" on Bloom's Taxonomy.

11.2 Agentic Assessment Example

11.2.1 Input

Educator Query	*"Please evaluate these questions for difficulty and fairness."*
Assessment Context	*"1. What is the definition of photosynthesis?* *2. Which of the following is NOT a property of water?"*

11.2.2 Agent Processing

1. **RAG Retrieval:** The FAISS retriever returns 3 chunks covering Bloom's Taxonomy guidelines, MCQ best practices (particularly the rule about avoiding negative phrasing like "NOT"), and difficulty calibration strategies.

2. **LLM Synthesis:** Gemini processes the retrieved pedagogy alongside the questions and generates a structured `AssessmentReport`.

11.2.3 Structured Output (Example)

Field	Content
`summary`	Both questions primarily test lower-order thinking skills. Question 1 targets "Remembering" and Question 2 uses negative phrasing, which may confuse students and test reading comprehension rather than domain knowledge.
`gaps`	• Q1 only tests factual recall (Bloom's Level 1). • Q2 uses "NOT" phrasing — violates MCQ best practices. • No questions target higher-order skills (Analyzing, Evaluating, Creating).
`advice`	• Reframe Q1 to application level: *"In which scenario would photosynthesis be the limiting factor?"* • Rewrite Q2 with positive framing: *"Which of the following is a unique property of water?"* • Add a question targeting "Evaluating" or "Creating" level.
`refs`	• Bloom's Taxonomy: Remembering vs. Applying levels. • MCQ Best Practices: "Avoid negative phrasing. If a negative must be used, capitalize and bold it."
`disclaimer`	This assessment was generated by an AI-powered pedagogy agent. Results should be reviewed by a qualified educator before implementation.

11.3 Comparative Analysis: ML vs. Agentic Approach

Dimension	ML Pipeline (Milestone 1)	Agentic AI (Milestone 2)
Output	Label + confidence %	Structured report with reasoning
Speed	< 0.1 seconds	$\sim$ 2–5 seconds
Explainability	None (black-box label)	Full reasoning chain with citations
Domain knowledge	Learned from data distribution	Retrieved from curated guidelines
Requires API	No	Yes (Gemini API key)
Offline capable	Yes	No (requires LLM API)
Use case	High-throughput batch classification	Deep qualitative assessment review

12 Limitations & Known Issues

12.1 ML Pipeline Limitations

1. **Post-Exam Dependency:** The model requires post-exam statistics (average score, correct rate, variance), limiting its use for pre-exam difficulty estimation.
2. **Text-Only Analysis:** The model cannot parse images, diagrams, or graphs embedded in questions.
3. **Language Support:** The TF-IDF vectoriser is optimised for English questions only.
4. **Single Model:** Only Logistic Regression is implemented; ensemble methods may yield improvements on more challenging datasets.
5. **Dataset Specificity:** Trained on 5,000 synthetic questions; real-world questions with unusual formatting may produce lower-confidence predictions.

12.2 Agentic Pipeline Limitations

1. **API Dependency:** The agent requires a valid `GEMINI_API_KEY` and internet connectivity.
2. **Latency:** The RAG + LLM pipeline introduces ~ 2.5 seconds of latency compared to < 0.1s for the ML classifier.
3. **Knowledge Base Size:** The pedagogy guidelines corpus is currently a single 3.1 KB document. Richer corpora (textbooks, standards documents) would improve retrieval quality.
4. **No Feedback Loop:** The agent does not learn from educator corrections — each invocation is stateless.
5. **Temperature Sensitivity:** While `temperature=0.2` reduces hallucination, it may also limit creative suggestions for question improvement.

13 Future Work

13.1 Short-Term Improvements

- Build a **pre-exam model** using only question text via BERT or sentence transformers.
- Compare against **ensemble methods** (XGBoost, Random Forest, SVM).
- Apply **SHAP values** for feature importance and model interpretability.
- Implement **hyperparameter tuning** via grid search or Bayesian optimisation.
- Expand the **RAG knowledge base** with additional educational standards (e.g., Webb's Depth of Knowledge, NCERT guidelines).
- Add **confidence thresholding** to flag low-confidence predictions for manual review.

13.2 Long-Term — Advanced Agentic Architecture

- **Multi-Agent System:** Separate agents for difficulty analysis, fairness auditing, Bloom's level classification, and question generation.
- **Conditional Graph Routing:** LangGraph edges that route dynamically based on question type (MCQ vs. free-response vs. numerical).
- **Multi-Modal RAG:** Vision-Language Models (e.g., Gemini Pro Vision) for diagram-dependent questions.
- **Iterative Feedback Loops:** Agentic workflows simulating student failure modes to refine difficulty estimates.
- **RAG Curriculum Alignment:** Retrieval against institutional curriculum standards for learning-outcome mapping.
- **Real-Time Model Updates:** Continuously update the ML model as new exam data arrives via online learning.

14 Conclusion

This project demonstrates a two-milestone approach to intelligent educational analytics:

Milestone 1 establishes that a straightforward combination of TF-IDF text vectorisation, numerical feature scaling, and Logistic Regression can achieve **98.9% accuracy** in predicting exam question difficulty. The balanced dataset of 5,000 questions across multiple subjects enables consistent performance across all three difficulty tiers. The scikit-learn `Pipeline` with `ColumnTransformer` encapsulates preprocessing and inference, eliminating feature-engineering leakage and simplifying deployment.

Milestone 2 extends the system with an **agentic AI workflow** that moves beyond classification to provide *qualitative, pedagogically-grounded feedback*. By combining **RAG** (FAISS + sentence-transformer embeddings over curated pedagogy literature) with **structured LLM generation** (Google Gemini via LangGraph), the system can identify assessment design flaws, recommend improvements aligned with Bloom's Taxonomy, and cite specific pedagogical principles — capabilities fundamentally beyond the reach of a traditional classifier.

Both milestones are unified in a production-ready **Streamlit web application** offering batch analytics, single-question prediction, and an AI-powered assessment design assistant. The system represents a practical blueprint for combining classical ML with modern generative AI in educational technology.

A Dependency Specification

The complete Python dependency list (`requirements.txt`):

Package	Category	Purpose
`streamlit`	Frontend	Web application
`pandas`	Data	DataFrame operations
`numpy`	Data	Numerical computation
`scikit-learn`	ML	Pipeline & classifier
`matplotlib`	Viz	Static charts
`seaborn`	Viz	Statistical plots
`joblib`	Util	Model serialisation
`langgraph`	Agent	Graph workflow engine
`langchain`	Agent	LLM abstraction layer
`langchain-core`	Agent	Core LangChain types
`langchain-community`	Agent	Community integrations
`langchain-google-genai`	Agent	Gemini LLM provider
`faiss-cpu`	RAG	Vector similarity search
`sentence-transformers`	RAG	Local embeddings
`pydantic`	Schema	Structured output
`python-dotenv`	Config	Environment variables

B Feature Vector Specification

The complete feature vector $\mathbf{x} \in \mathbb{R}^{3,003}$:

Index	Feature	Extractor	Dim
1–3,000	TF-IDF word features	`TfidfVectorizer`	3,000 (sparse)
3,001	`average_score`	`StandardScaler`	1
3,002	`correct_rate`	`StandardScaler`	1
3,003	`score_variance`	`StandardScaler`	1

C Pedagogy Knowledge Base Contents

The RAG retrieval corpus (`data/pedagogy_guidelines.txt`) covers four domains:

1. **Bloom's Taxonomy (6 levels):** Remembering, Understanding, Applying, Analyzing, Evaluating, Creating — with associated action verbs for question construction.
2. **MCQ Best Practices:** Clear stems, plausible distractors, consistent option length, avoidance of "All of the above"/"None of the above", no grammatical clues.

3. **Difficulty Remediation:** Strategies for overly Hard questions (simplify language, add prerequisites) and overly Easy questions (promote higher-order thinking, improve distractors).

4. **Inclusivity & Fairness:** Culturally neutral naming, avoidance of idioms and socio-economic assumptions, accessible scenarios.

D Reproduction Instructions

```
# 1. Clone the repository
git clone https://github.com/Siddharth-Dangi/\
  AI-driven_educational_analytics_system.git
cd AI-driven_educational_analytics_system

# 2. Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Build the FAISS index (one-time)
python -m agent.rag

# 5. Train the model (or run notebook/train.ipynb)
python retrain.py

# 6. Set Gemini API key (for Agent tab)
echo "GEMINI_API_KEY=your-key-here" > .env

# 7. Launch the application
streamlit run app.py
# Opens at http://localhost:8501
```

Listing 10: Full reproduction steps.

E Glossary

Term	Definition
Agentic Workflow	A directed graph of autonomous processing nodes that read from and write to a shared state, enabling multi-step LLM reasoning.
RAG	Retrieval-Augmented Generation: a technique that grounds LLM output in externally retrieved documents to reduce hallucination and improve factual accuracy.
FAISS	Facebook AI Similarity Search: an efficient library for nearest-neighbour search over dense embedding vectors.
TF-IDF	Term Frequency–Inverse Document Frequency: a numerical statistic reflecting the importance of a word in a document relative to a corpus.
LangGraph	A framework for building stateful, graph-based LLM applications with explicit state management and composable nodes.
Structured Output	A technique where the LLM is constrained to produce output matching a predefined schema (e.g., Pydantic model), eliminating free-form text.
Bloom's Taxonomy	A hierarchical framework classifying cognitive skills into six levels, from Remembering to Creating.